

Documento de arquitectura de software del sistema ¿Quién lo tiene?: estilo arquitectónico, vistas y justificación de las decisiones técnicas

Daniel Montenegro Almendra
Brayan Grajales Escarraga
Monica Anyi Cajas Mellizo
Yanimar Burbano Piamba
Wilson Javier Galindez Certuche
Francisco José Castro Muñoz
Deiner Stiven Ramírez Vente

Equipo Byte Force

Fundación Universitaria de Popayán

Facultad de Ingeniería de Sistemas

Ingeniería de Software III

Docente: Andres Felipe Arboleda

30 de octubre de 2026

Tabla de contenido

Introducción....3
Objetivos....3
Objetivo General....3
Objetivos Específicos....4
Enfoque de la Descripción Arquitectónica....4
Impulsores Arquitectónicos....5
Requisitos del Negocio que Moldean la Arquitectura....6
Atributos de Calidad Prioritarios....6
Restricciones....7
Decisiones Arquitectónicas y su Justificación....8
Estilo Arquitectónico: Monolito Modular....8
Organización Interna: Arquitectura Hexagonal por Módulo....9
Lenguaje y Framework: Java 21 con Spring Boot 4....10
Base de Datos: PostgreSQL 17 con PostGIS....11
Frontend: React con Vite como Aplicación Web Progresiva....12
Comunicación en Tiempo Real: SSE y Push Web....12
Comunicación entre Módulos: Eventos de Dominio en el Mismo Proceso....12
Despliegue: Docker Compose y Caddy en un VPS....13
Autenticación y Sesión....13
Resumen de Decisiones....14
Vista Lógica....14
Módulos de Negocio....15
Reglas del Monolito Modular....17
Vista de Desarrollo....18
Vista de Procesos....20
Vista de Datos....22
Vista Física y Despliegue....23
Diseño de la API REST....25
Patrones de Diseño y Principios SOLID....26
Seguridad....27
Rendimiento y Optimización....28
Pila Tecnológica y Licencias....29
Verificación de la Arquitectura....30
Responsabilidades del Equipo sobre la Arquitectura....32
Riesgos Arquitectónicos y Evolución....33
Conclusiones....33
Referencias....35
Apéndice....37

Introducción

¿Quién lo tiene? es una aplicación web progresiva (PWA) en la que un cliente publica el producto que necesita y las tiendas cercanas de la categoría responden si lo tienen, a qué precio y si hacen domicilio. El cliente compara las respuestas, elige una y la elección se convierte en una orden para recoger en la tienda o recibir a domicilio. Funciona como una subasta inversa local y en tiempo real, no como un comercio electrónico con catálogo, y esa diferencia explica la mayoría de las decisiones de este documento.

El documento describe la arquitectura de software con la que el equipo Byte Force construye el producto mínimo viable (MVP) que se entrega el 26 de noviembre de 2026. Complementa el plan de proyecto, que fijó el ciclo de vida híbrido Scrum con prácticas DevOps, los roles y los requerimientos, y la Especificación de Requisitos de Software (SRS), que detalla los requisitos, las historias de usuario y la trazabilidad. Aquí se explica cómo se organiza el sistema para cumplir esos requisitos y, sobre todo, por qué se eligió cada tecnología y cada estilo frente a sus alternativas.

La arquitectura elegida es un **monolito modular** en Java 21 con Spring Boot 4, con arquitectura hexagonal (puertos y adaptadores) dentro de cada módulo, una base de datos PostgreSQL con la extensión PostGIS para las búsquedas por radio, un frontend en React instalable como PWA y un despliegue con Docker Compose y Caddy en un único servidor virtual de bajo costo. Toda la pila usa licencias que permiten el uso comercial sin pagar, porque el proyecto debe salir con el menor costo posible.

Objetivos

Objetivo General

Definir y justificar la arquitectura de software de ¿Quién lo tiene? de manera que el MVP funcione en un servidor de recursos limitados, cumpla los requisitos de seguridad y privacidad, y pueda crecer sin reescribirse.

Objetivos Específicos

- Identificar los impulsores arquitectónicos: los requisitos del negocio, los atributos de calidad prioritarios y las restricciones del proyecto.
- Comparar las alternativas de estilo arquitectónico, lenguaje, base de datos, frontend, comunicación en tiempo real y despliegue, y justificar la elección de cada una.
- Describir el sistema mediante vistas lógica, de desarrollo, de procesos, de datos y física.
- Establecer los patrones de diseño, los controles de seguridad, las medidas de optimización y la estrategia de verificación de la arquitectura.
- Asignar las responsabilidades del equipo sobre la arquitectura e identificar sus riesgos y su evolución.

Enfoque de la Descripción Arquitectónica

La norma ISO/IEC/IEEE 42010:2022 establece que una descripción de arquitectura debe identificar a los interesados, sus preocupaciones y un conjunto de vistas que respondan a esas preocupaciones (International Organization for Standardization [ISO] et al., 2022). Este documento sigue esa norma y organiza las vistas a partir del modelo 4+1 de Kruchten (1995), con una vista de datos adicional porque la base geoespacial es central en el producto. Los atributos de calidad se nombran según el modelo de la norma ISO/IEC 25010:2023 y las decisiones se registran con un formato breve de decisiones arquitectónicas: contexto, alternativas, decisión y consecuencias (Bass et al., 2021).

Tabla 1

Vistas de la Arquitectura y Preocupaciones que Atienden

Vista	Pregunta que responde	Interesados principales	Diagramas relacionados
Lógica	¿Qué módulos de negocio hay y cómo se comunican?	Arquitecto, desarrolladores, Product Owner	Componentes (07), clases de dominio (02)
Desarrollo	¿Cómo se organiza el código y qué reglas de dependencia lo protegen?	Desarrolladores, QA	Figura 3, estructura de paquetes
Procesos	¿Qué ocurre en ejecución cuando se publica, responde y elige?	Desarrolladores, QA	Secuencias (03, 04), estados (05), actividades (06)
Datos	¿Dónde y cómo se guarda la información y quién es su dueño?	Desarrolladores backend, docente	Entidad-relación (09)
Física	¿En qué infraestructura corre y cómo llega a producción?	Arquitecto, Scrum Master	Despliegue (08), Figura 6
Escenarios (+1)	¿Qué casos de uso validan las demás vistas?	Todos	Contexto (00), casos de uso (01)

Nota. Los números entre paréntesis corresponden a los archivos .drawio de la carpeta `docs/diagramas` del repositorio, listados en el Apéndice. Elaboración propia del equipo Byte Force.

Impulsores Arquitectónicos

Los impulsores arquitectónicos son los requisitos que más condicionan la estructura del sistema. Se agrupan en tres tipos: lo que exige el negocio, los atributos de calidad que deben cumplirse y las restricciones que no se pueden cambiar.

Requisitos del Negocio que Moldean la Arquitectura

Tabla 2

Requisitos del Negocio y su Consecuencia Técnica

Requisito del negocio	Consecuencia técnica
La solicitud llega solo a tiendas abiertas, de la categoría y a menos de 3 km; si nadie responde en 10 minutos el radio se amplía una vez a 5 km.	Consultas geoespaciales con índice (PostGIS) y tareas programadas que reintentan con un radio mayor.
La tienda recibe un aviso y el cliente ve las respuestas a medida que llegan.	Canal en tiempo real (SSE) mientras la app está abierta y notificaciones push web cuando está cerrada.
La solicitud vence a los 30 minutos y la reserva de la orden entre 30 y 60 minutos.	Máquinas de estado con vencimiento y tareas programadas livianas.
La tienda no ve el nombre ni la dirección del cliente hasta que la eligen, y la dirección se oculta tras la entrega.	Autorización a nivel de objeto y DTO de salida distintos por rol, por la Ley 1581 de 2012.
Las tiendas se verifican con documentos y fotos; los productos restringidos no admiten domicilio.	Almacenamiento de archivos detrás de un puerto reemplazable y reglas de dominio explícitas.
Máximo 10 solicitudes por hora por cliente.	Límites de uso en memoria por usuario.
Las reseñas solo se permiten sobre órdenes resueltas.	Reglas en un módulo de reputación que reacciona a los eventos de las órdenes.
El cobro dentro de la app queda fuera de esta entrega, pero se hará en el futuro.	Un puerto `PasarelaPago` definido desde ahora, sin implementación real.
El arranque es una demostración con datos de prueba y luego un piloto de 10 a 15 tiendas.	Un monolito en un solo servidor es suficiente; la complejidad distribuida no se justifica.

Nota. Valores aprobados por el equipo en el levantamiento de requisitos (Fase 9 de la SRS).

Atributos de Calidad Prioritarios

Se priorizaron cinco características de ISO/IEC 25010:2023. Las metas de la Tabla 3 son objetivos de diseño que se comprueban con las pruebas descritas más adelante; las métricas definitivas de aceptación están en los requerimientos no funcionales de la SRS.

Tabla 3

Atributos de Calidad Prioritarios y Tácticas Arquitectónicas

Atributo (ISO/IEC 25010)	Escenario	Meta de diseño	Tácticas principales
Eficiencia de desempeño	Varios clientes publican y decenas de tiendas responden a la vez.	API, base y proxy caben en el servidor con margen de memoria; respuestas de lectura rápidas para el usuario.	Presupuesto de memoria, hilos virtuales, índices GiST, caché en memoria, proyecciones a DTO.
Seguridad	Una tienda intenta ver datos de un cliente que no la eligió.	El intento se rechaza siempre y queda auditado.	Autorización por objeto, DTO por rol, OWASP ASVS 5.0 nivel 2.
Fiabilidad	La API se reinicia justo después de guardar una respuesta.	El aviso al cliente no se pierde.	Eventos con registro persistente (outbox) y reintento.
Mantenibilidad	Se agrega el cobro en línea o una nueva categoría.	El cambio queda dentro de un módulo y de sus adaptadores.	Monolito modular, puertos y adaptadores, pruebas de fronteras.
Flexibilidad (escalabilidad)	El piloto pasa de una zona a varias ciudades.	Crecer sin reescribir: más recursos, réplicas o extracción de un módulo.	Módulos con fronteras verificadas, un esquema de base por módulo.

Nota. Elaboración propia del equipo Byte Force con base en ISO/IEC 25010:2023.

Restricciones

- **Presupuesto.** El único costo fijo aceptado es el servidor y el dominio. El servidor de producción es un VPS Hostinger KVM 1 (COP 57.900); según la oferta del proveedor tiene 1 vCPU y 4 GB de RAM, dato que se confirma al contratar. La arquitectura se diseñó para caber en 2 GB y dejar el resto como margen.
- **Licencias.** Solo componentes gratuitos que permitan uso personal y comercial, sin costo de licenciamiento.
- **Plazo y equipo.** Cuatro sprints semanales (del 26 de octubre al 22 de noviembre) y siete integrantes, de los cuales tres desarrollan.
- **Tecnologías del curso.** Java con Spring Boot en el backend y React en el frontend, como definió el equipo al inicio del proyecto.

- **Legales.** Ley 1581 de 2012 de protección de datos personales: política de tratamiento, autorización explícita al registrarse y minimización de datos.

Decisiones Arquitectónicas y su Justificación

Esta sección responde por qué se eligió cada pieza. Para cada decisión se presentan las alternativas que se evaluaron, los criterios y la razón de la elección. Ninguna alternativa descartada es mala en general: se descartó porque no se ajusta a las restricciones de este proyecto.

Estilo Arquitectónico: Monolito Modular

Se compararon cuatro estilos con siete criterios del proyecto, calificados de 1 a 5, donde 5 es el mejor ajuste. La Tabla 4 resume el resultado.

Tabla 4

Matriz Comparativa de Estilos Arquitectónicos

Estilo	CI	RE	TE	MA	ES	CO	EQ	Total
Monolito modular	5	5	5	4	4	5	5	**33**
Monolito en capas tradicional	5	5	5	2	2	5	5	29
Funciones sin servidor (serverless)	3	4	2	3	5	2	2	21
Microservicios	1	1	1	4	5	1	2	15

Nota. CI = costo de infraestructura; RE = consumo de recursos del servidor; TE = tiempo de entrega en 4,5 semanas; MA = mantenibilidad y modularidad; ES = escalabilidad futura; CO = complejidad operativa (5 = más simple); EQ = ajuste al tamaño y experiencia del equipo. Escala de 1 a 5; total máximo 35. Elaboración propia.

Por qué no microservicios. Los microservicios resuelven problemas de organizaciones con muchos equipos que despliegan por separado y de cargas que escalan de forma muy desigual (Newman, 2021). Este proyecto tiene un equipo pequeño, un solo servidor y un piloto de pocas tiendas. Con diez servicios, cada uno con su JVM, se necesitarían varios gigabytes de RAM solo para arrancar, además de un broker de mensajes, descubrimiento de servicios y trazas distribuidas. Las transacciones entre solicitudes y órdenes se volverían sagas distribuidas. El

costo y la complejidad superan cualquier beneficio en el MVP, y Fowler (2015) recomienda justamente empezar por un monolito y separar servicios solo cuando las fronteras estén probadas.

Por qué no un monolito en capas tradicional. Un monolito con paquetes globales `controller`, `service` y `repository` es igual de barato, pero con el tiempo cualquier clase termina usando cualquier otra y el sistema se vuelve difícil de cambiar. El monolito modular conserva el bajo costo de un solo despliegue y agrega fronteras de negocio que se verifican con pruebas automáticas, de modo que un módulo pueda extraerse como servicio el día que haga falta.

Por qué no funciones sin servidor. Las funciones tienen costo bajo con poco tráfico, pero no admiten bien conexiones abiertas de larga duración como SSE, el arranque en frío de Java es lento y el sistema quedaría atado a un proveedor de nube específico, contra la decisión de portabilidad con Docker.

Organización Interna: Arquitectura Hexagonal por Módulo

Dentro de cada módulo se aplica la arquitectura hexagonal de Cockburn (2005), que comparte la regla de dependencia de Clean Architecture (Martin, 2017): el dominio no depende de nada técnico y la infraestructura depende del dominio. La regla se aplica por módulo y no como capas globales del sistema, para que cada módulo sea una unidad completa.

Tabla 5

Comparación de Organizaciones Internas del Código

Opción	Ventaja	Desventaja para este proyecto	Decisión
Hexagonal por módulo (puertos y adaptadores)	El dominio se prueba sin base de datos; cambiar disco por S3 o agregar Wompi no toca los casos de uso.	Más paquetes y algo de mapeo entre capas.	**Elegida**, con mapeo automático (MapStruct) para reducir el costo.
Capas MVC clásicas (controller, service, repository)	Muy conocida y rápida al inicio.	La lógica de negocio queda mezclada con JPA y HTTP; las fronteras entre módulos no existen.	Descartada.
Clean Architecture estricta con capas globales	Separación máxima.	Duplica clases sin motivo y separa por capa técnica, no por negocio.	Descartada; se toma solo su regla de dependencia.

Lenguaje y Framework: Java 21 con Spring Boot 4

Tabla 6

Comparación de Plataformas para el Backend

Plataforma	Fortalezas	Limitaciones para el proyecto	Decisión
Java 21 LTS + Spring Boot 4	Lenguaje del curso y del equipo; Spring Security, Spring Data JPA con Hibernate Spatial y Spring Modulith para verificar módulos; hilos virtuales; licencias Apache 2.0.	Consume más memoria que Node.js o Go; se controla con heap fijo y, si hace falta, imagen nativa con GraalVM.	**Elegida**
Node.js + NestJS	Menor consumo de memoria; mismo lenguaje que el frontend.	El equipo no lo domina; el ecosistema geoespacial y de seguridad es menos integrado; no hay verificación de módulos equivalente.	Descartada
C# + ASP.NET Core	Muy buen rendimiento; licencia MIT.	Fuera del curso; el equipo tendría que aprenderlo durante el plazo de 4,5 semanas.	Descartada
Python + Django o FastAPI	Desarrollo rápido; GeoDjango es sólido.	Menor rendimiento por proceso y tipado débil para un dominio con muchas reglas; fuera del curso.	Descartada

Nota. El JDK es Eclipse Temurin, gratuito para uso comercial; se evita Oracle JDK por su licencia.

La razón principal es el riesgo: aprender una plataforma nueva en cuatro semanas pondría en peligro la entrega, mientras que el mayor costo de memoria de Java se resuelve con configuración. Spring Boot 4 además ofrece de forma gratuita lo que este sistema necesita: seguridad con JWT, persistencia geoespacial, tareas programadas, SSE y eventos de dominio con registro persistente.

Base de Datos: PostgreSQL 17 con PostGIS

Tabla 7

Comparación de Motores de Base de Datos

Motor	Soporte geoespacial	Licencia y costo	Decisión
PostgreSQL + PostGIS	Completo: tipo `geography`, índice GiST y `ST_DWithin` para distancias reales en metros.	PostgreSQL License y GPLv2 (PostGIS); gratis para uso comercial como servidor.	**Elegida**
MySQL 8 Community	Tipos espaciales e índices R-tree, con menos funciones de distancia y geografía.	GPLv2 con licencia dual de Oracle.	Descartada por un soporte geoespacial más limitado.
MongoDB	Bueno: índices `2dsphere`.	SSPL desde 2018, no es una licencia de código abierto reconocida por la OSI.	Descartada: licencia y modelo no relacional para órdenes con transacciones.
SQL Server Express	Tipo `geography`.	Propietaria; límite de 10 GB por base y de memoria.	Descartada

PostgreSQL resuelve además, sin servicios adicionales, la búsqueda de texto de productos con `pg_trgm` y `unaccent`, en lugar de Elasticsearch, y la coordinación de tareas programadas con `FOR UPDATE SKIP LOCKED`. Una sola base con un esquema por módulo mantiene la separación lógica sin pagar por varias instancias. PostgreSQL escala bien en vertical y admite réplicas de lectura cuando sea necesario.

Frontend: React con Vite como Aplicación Web Progresiva

Tabla 8

Comparación de Opciones para el Cliente

Opción	Ventaja	Desventaja para el proyecto	Decisión
React + Vite como PWA	Un solo código para celular y computador; se instala desde el navegador; notificaciones push web sin tiendas de aplicaciones.	En iPhone el push web solo funciona con la app instalada en la pantalla de inicio (iOS 16.4 o posterior).	**Elegida**
Apps nativas (Kotlin y Swift)	Mejor integración con el sistema.	Dos códigos; cuenta de Apple de 99 USD al año y de Google de 25 USD; revisión de tiendas.	Descartada por costo y plazo
Flutter o React Native	Un código para dos plataformas nativas.	Igual requiere publicar en tiendas; el equipo no lo domina.	Descartada
Angular	Framework completo.	Más pesado y fuera de la experiencia del equipo; React tiene más librerías de mapas y PWA.	Descartada

Nota. Create React App está abandonado, por eso se usa Vite. El mapa usa Leaflet o MapLibre con teselas de OpenStreetMap.

Comunicación en Tiempo Real: SSE y Push Web

El servidor necesita avisar al cliente cuando llega una respuesta y a la tienda cuando llega una solicitud; el usuario casi nunca envía datos por ese canal. Por eso se eligió **Server-Sent Events (SSE)**: funciona sobre HTTP normal, pasa sin configuración por Caddy, se reconecta solo y, con hilos virtuales, cada conexión abierta cuesta muy poca memoria. WebSocket se descartó porque su comunicación bidireccional no hace falta y complica el proxy y la autenticación; el sondeo periódico se descartó porque multiplica las peticiones al servidor sin aportar inmediatez. Cuando la app está cerrada se usa **push web estándar con VAPID**, gratuito en los navegadores; Firebase Cloud Messaging se descartó porque ata el sistema a Google y no es necesario en la web.

Comunicación entre Módulos: Eventos de Dominio en el Mismo Proceso

Los módulos se comunican de dos formas: consultas síncronas a la fachada `api` de otro módulo cuando necesitan una respuesta inmediata, y **eventos de dominio** para las reacciones

secundarias (avisar, recalcular la reputación, habilitar la reseña). Los eventos se publican con Spring Modulith, que guarda cada publicación en la base de datos en la misma transacción y la reintenta si la aplicación se cae (patrón outbox transaccional). Un broker externo como Kafka o RabbitMQ se descartó porque agrega un proceso de cientos de megabytes y operación adicional sin resolver un problema que el MVP tenga; si un módulo se extrae en el futuro, sus eventos pueden pasar a un broker sin cambiar el dominio.

Despliegue: Docker Compose y Caddy en un VPS

Tabla 9

Comparación de Opciones de Despliegue

Opción	Ventaja	Desventaja para el proyecto	Decisión
Docker Compose en un VPS	Costo fijo bajo; mismo entorno en desarrollo y producción; despliegue con un comando.	Un solo servidor es un punto único de falla; se mitiga con copias de seguridad probadas.	**Elegida**
Kubernetes (k3s o administrado)	Alta disponibilidad y escalado automático.	Consume buena parte de la RAM del servidor solo en el plano de control; complejidad alta para el equipo.	Descartada
Plataforma como servicio (PaaS)	Sin administrar servidor.	Los planes gratuitos se suspenden por inactividad o limitan la base de datos; los pagos superan al VPS.	Descartada
Instalación directa sin contenedores	Sin capa adicional.	Entornos distintos entre integrantes; actualizaciones manuales y propensas a errores.	Descartada

Nota. Caddy se prefirió frente a Nginx porque obtiene y renueva automáticamente los certificados TLS de Let's Encrypt con una configuración de pocas líneas; Nginx requiere Certbot y más configuración. Docker Engine y Compose en Linux son gratuitos.

Autenticación y Sesión

Se usa Spring Security con un token de acceso JWT de 15 minutos y un token de refresco opaco y rotativo, guardado como hash en la base y enviado en una cookie `HttpOnly`, `Secure` y `SameSite=Strict`. Todos los usuarios entran con una frase de contraseña de mínimo 15 caracteres, como exige NIST SP 800-63B-4 cuando la contraseña es el único factor (Temoshok

et al., 2025), y el administrador además usa TOTP. Se descartó Keycloak porque es un proceso adicional de varios cientos de megabytes de RAM, y los proveedores externos como Auth0 porque tienen costo a partir de cierto volumen. El inicio de sesión con código enviado al correo queda como mejora futura.

Resumen de Decisiones

Tabla 10

Registro de Decisiones Arquitectónicas

ID	Decisión	Alternativa principal descartada	Razón clave
DA-01	Monolito modular	Microservicios	Costo y memoria de un solo servidor; equipo pequeño.
DA-02	Hexagonal dentro de cada módulo	Capas MVC globales	Dominio independiente de la tecnología y fronteras claras.
DA-03	Java 21 y Spring Boot 4	Node.js con NestJS	Experiencia del equipo y ecosistema integrado.
DA-04	PostgreSQL 17 con PostGIS	MySQL 8	Búsquedas por radio con distancias reales e índice GiST.
DA-05	Un esquema por módulo con Flyway	Una base por módulo	Separación lógica sin costo de varias instancias.
DA-06	React con Vite como PWA	Apps nativas	Un solo código y sin costos de tiendas.
DA-07	SSE y push web con VAPID	WebSocket y Firebase	Comunicación del servidor al cliente, simple y sin dependencias.
DA-08	Eventos de dominio con outbox	Kafka o RabbitMQ	Fiabilidad sin procesos adicionales.
DA-09	Docker Compose y Caddy en VPS	Kubernetes	Operación simple y TLS automático.
DA-10	JWT corto, refresco opaco, frase de 15 caracteres y TOTP	Keycloak	Cumple NIST 800-63B-4 sin otro proceso.
DA-11	Caché en memoria con Caffeine	Redis	Sin proceso adicional; Redis cambió su licencia.

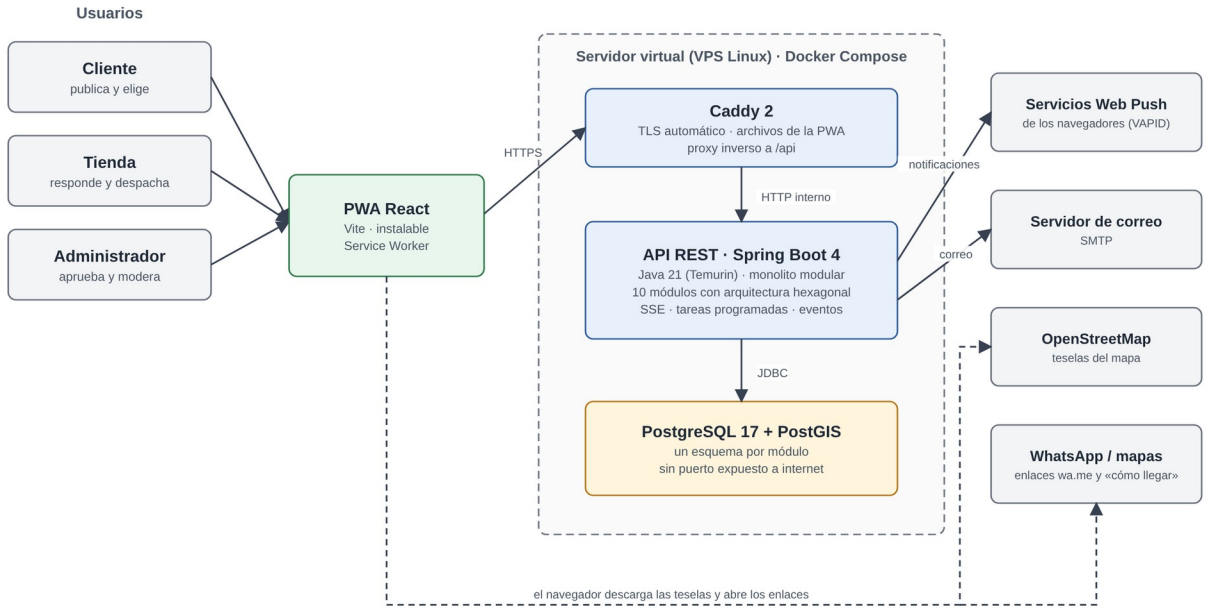
Nota. Elaboración propia del equipo Byte Force.

Vista Lógica

La Figura 1 muestra el sistema en su contexto: los tres roles usan la PWA, todo el tráfico entra por Caddy y la API es la única que habla con la base de datos. Los servicios externos son gratuitos y ninguno es imprescindible para el flujo principal.

Figura 1

Vista de Contexto y Contenedores del Sistema



Nota. El mapa y los enlaces de WhatsApp y «cómo llegar» se resuelven en el navegador, por lo que no consumen recursos del servidor ni requieren claves de API. Elaboración propia.

Módulos de Negocio

El backend se divide en diez módulos de negocio y un núcleo compartido mínimo. El corazón del producto son `solicitudes` y `ordenes`; los demás los rodean. La separación entre ambos es deliberada: una solicitud recibe muchas respuestas y casi todas se descartan, mientras que la orden es el compromiso real con una sola tienda y tiene su propio ciclo de vida.

Tabla 11

Módulos de Negocio del Backend

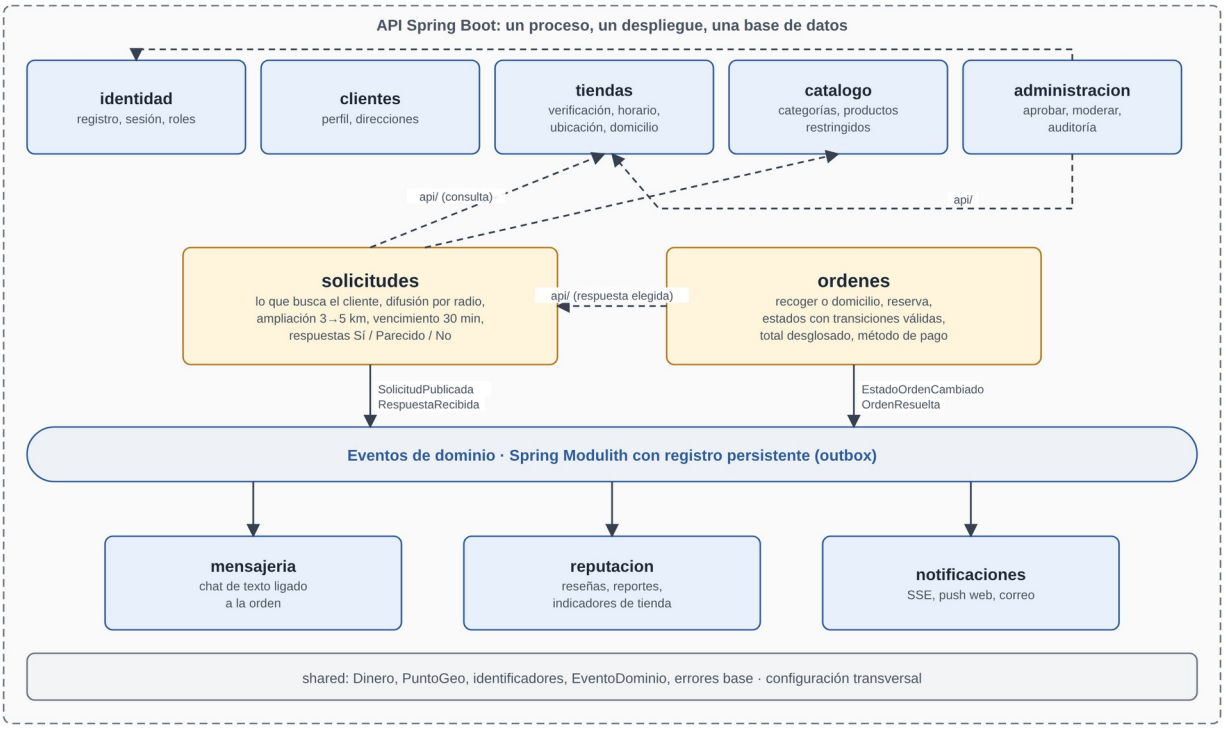
Módulo	Responsabilidad	Agregados principales	Eventos que publica
`identidad`	Registro, inicio de sesión, roles, tokens, verificación de correo	Usuario, Credencial, TokenRefresco	UsuarioRegistrado, UsuarioSuspendido
`clientes`	Perfil del cliente y direcciones guardadas	Cliente, Direccion	—
`tiendas`	Registro y verificación, horario, interruptor abierto o cerrado, ubicación, zonas y tarifas de domicilio, métodos de pago	Tienda, ZonaDomicilio, TarifaDomicilio	TiendaAprobada, TiendaAbierta, TiendaCerrada
`catalogo`	Categorías activas y productos restringidos	Categoria, ProductoRestringido	—
`solicitudes`	Publicación, difusión por radio, ampliación, vencimiento y respuestas Sí, Parecido o No	Solicitud, Respuesta	SolicitudPublicada, RespuestaRecibida
`ordenes`	Recoger o domicilio, reserva, estados, total desglosado y método de pago	Orden	EstadoOrdenCambiado, OrdenResuelta
`mensajeria`	Mini chat de texto ligado a la orden	Conversacion, Mensaje	MensajeEnviado
`reputacion`	Reseñas de órdenes resueltas, reportes e indicadores de la tienda	Resena, Reporte, IndicadoresTienda	ReporteCreado
`notificaciones`	Escucha eventos y avisa por SSE, push web o correo	SuscripcionPush, Notificacion	—
`administracion`	Aprobar tiendas, revisar reportes, suspender cuentas y auditar	AccionAdministrativa	—
`shared`	Tipos comunes sin lógica de negocio: Dinero, PuntoGeo, identificadores, errores base	—	—

Nota. Los nombres de los paquetes no llevan tildes para evitar problemas de codificación.

Elaboración propia.

Figura 2

Módulos del Monolito y su Comunicación



Nota. Las flechas punteadas son consultas a la fachada `api` de otro módulo; las continuas, publicación y escucha de eventos de dominio. Ningún módulo lee tablas de otro. Elaboración propia.

Reglas del Monolito Modular

1. Cada módulo es dueño de sus tablas, en su propio esquema de PostgreSQL. Ningún módulo lee ni escribe tablas de otro.
2. Un módulo solo usa a otro a través de su paquete `api` (una fachada con DTO) o escuchando sus eventos.
3. Las reacciones secundarias van por eventos asíncronos dentro del mismo proceso, con registro persistente para no perderlos.
4. No hay dependencias circulares entre módulos.

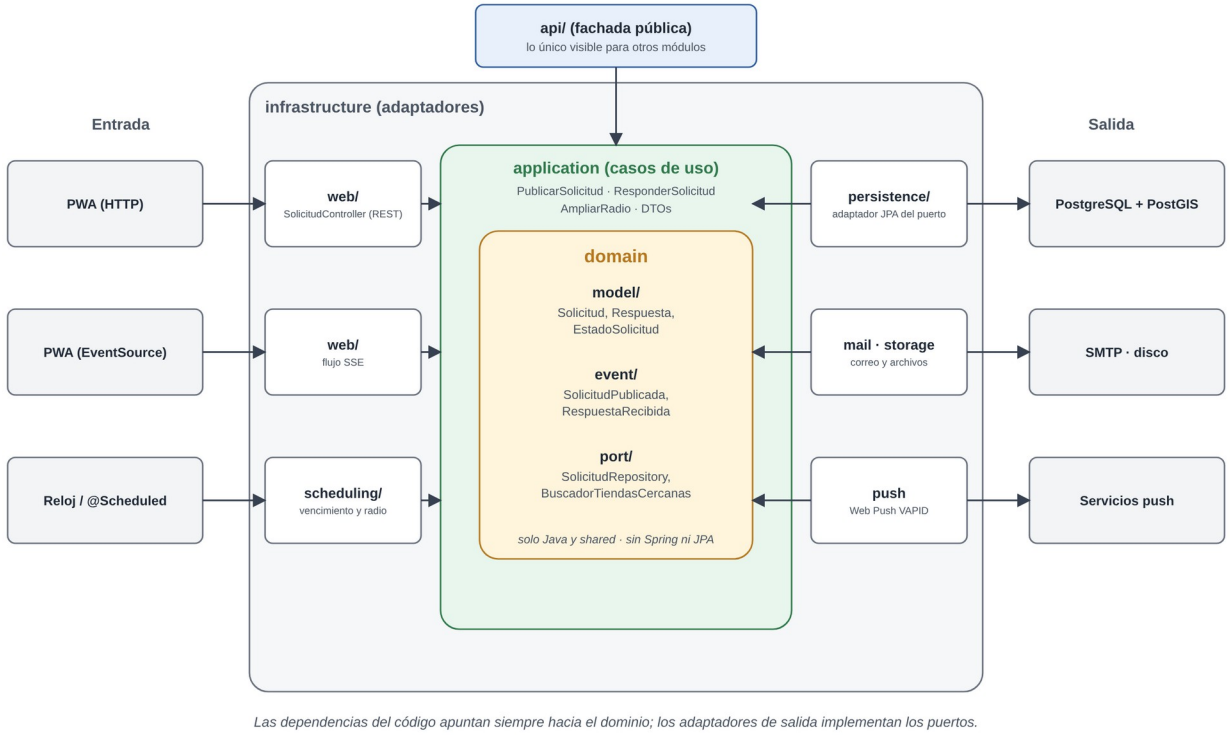
5. Dentro de cada módulo se respeta la regla de dependencia: la infraestructura depende de la aplicación y esta del dominio, nunca al revés.

Vista de Desarrollo

Todo el backend es un solo proyecto Maven con un paquete por módulo. Es más simple que un build de varios módulos de Maven y las fronteras se hacen cumplir con pruebas automáticas en lugar del compilador. La Figura 3 muestra la estructura de un módulo.

Figura 3

Arquitectura Hexagonal dentro del Módulo Solicitudes



Nota. Los adaptadores de entrada llaman a los casos de uso; los de salida implementan los puertos que define el dominio. Elaboración propia con base en Cockburn (2005).

Tabla 12

Capas de cada Módulo y Reglas de Dependencia

Paquete	Contenido	Puede depender de
`domain/model`	Entidades, objetos de valor, máquinas de estado	Solo Java y `shared`
`domain/event`	Eventos de dominio	Solo Java y `shared`
`domain/port`	Interfaces Repository y de servicios externos	Solo Java y `shared`
`application/usecase`	Un caso de uso por acción del usuario y control de transacciones	`domain`
`application/dto`	DTO de entrada y salida	`domain`
`infrastructure/web`	Controladores REST (`/api/v1/...`) y flujos SSE	`application`, `domain`
`infrastructure/persistence`	Entidades JPA y adaptadores Spring Data	`application`, `domain`
`api`	Fachada pública y eventos que el módulo expone a los demás	`application`

Nota. Tomado de `docs/ESTRUCTURA.md` del repositorio.

La estructura de paquetes del módulo `solicitudes`, que se repite en los demás, es la siguiente:

```
com.example.quienlotiene
├── QuienLoTieneApplication.java
├── shared/              Dinero, PuntoGeo, ids, EventoDominio, errores
├── solicitudes/
│   ├── api/             SolicitudesApi, eventos publicados
│   ├── domain/
│   │   ├── model/       Solicitud, Respuesta, EstadoSolicitud
│   │   ├── event/       SolicitudPublicada, RespuestaRecibida
│   │   └── port/        SolicitudRepository, BuscadorTiendasCercanas
│   ├── application/
│   │   ├── usecase/     PublicarSolicitud, ResponderSolicitud, AmpliarRadio
│   │   └── dto/
│   └── infrastructure/
│       ├── web/         SolicitudController, mappers
│       └── persistence/ entidades JPA, adaptadores del puerto
├── ordenes/  tiendas/  identidad/  clientes/  catalogo/
└── mensajeria/  reputacion/  notificaciones/  administracion/
```

Las reglas se verifican en cada compilación:

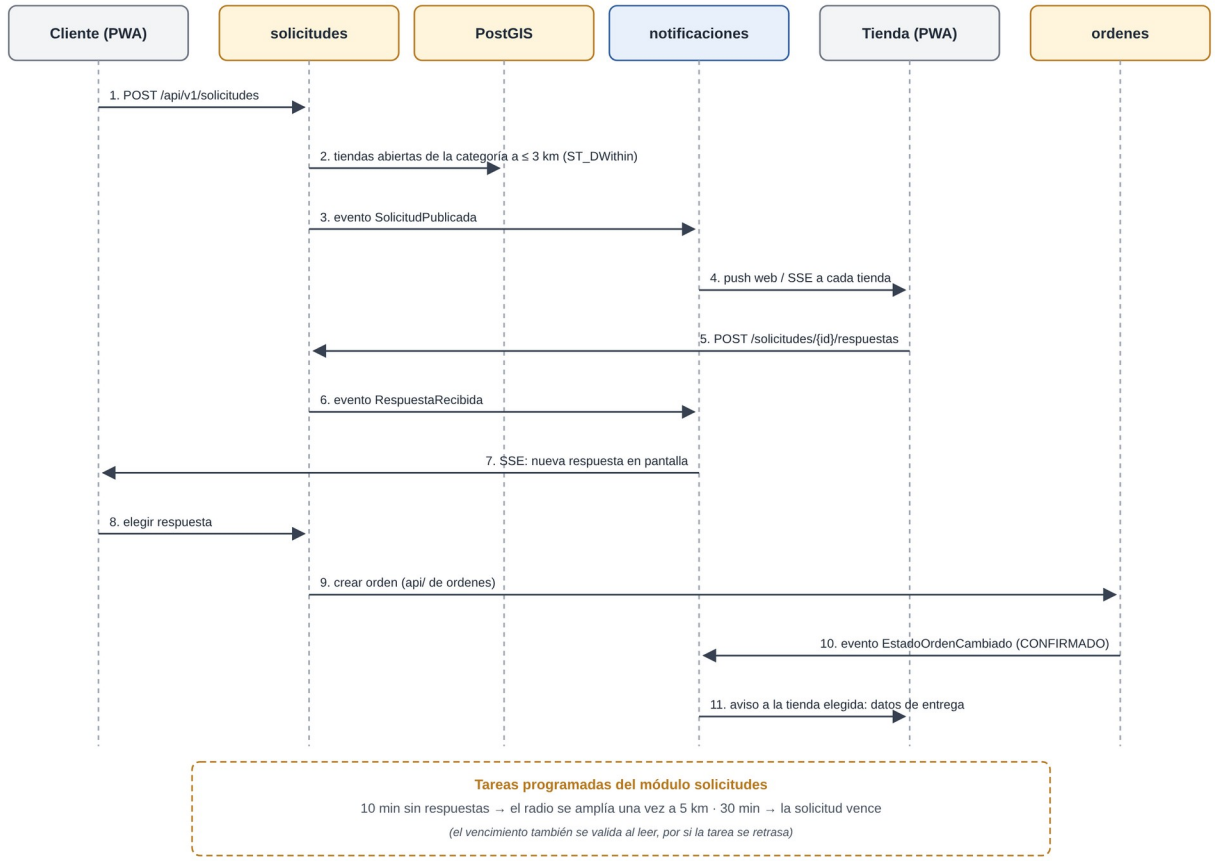
- **Spring Modulith:** una prueba con `ApplicationModules.verify()` falla si un módulo usa algo que no está en la `api` de otro o si hay ciclos, y genera la documentación de los módulos.
- **ArchUnit:** pruebas que fallan si `domain` importa Spring, JPA o algo de `infrastructure`.
- **MapStruct:** convierte entre dominio, JPA y DTO con código generado al compilar, sin reflexión, lo que ahorra CPU y memoria.
- **Flyway:** migraciones versionadas en `src/main/resources/db/migration/<modulo>/`, una carpeta y un esquema por módulo.

Vista de Procesos

La Figura 4 resume el flujo principal en ejecución, desde que el cliente publica la solicitud hasta que la tienda elegida recibe la orden. El detalle completo está en los diagramas de secuencia 03 y 04.

Figura 4

Flujo Principal en Ejecución: Publicar, Responder y Elegir



Nota. La tienda ve la solicitud sin datos del cliente; solo la tienda elegida recibe los datos de entrega. Basado en los diagramas de secuencia 03 y 04 del proyecto. Elaboración propia.

Los estados se implementan con el patrón State mediante un `enum` que conoce sus transiciones válidas, lo que es más liviano que una clase por estado. La solicitud pasa de ABIERTA a RESUELTA, cuando el cliente elige, o a VENCIDA a los 30 minutos sin elección. La orden pasa de CONFIRMADO a PREPARANDO y luego a EN CAMINO, si es domicilio, o a LISTO PARA RECOGER; después a ENTREGADO y a RESUELTA cuando el cliente confirma la recepción. También puede quedar CANCELADO o VENCIDA si la reserva expira. Cualquier salto inválido, como pasar de CONFIRMADO a ENTREGADO, se rechaza.

Las tareas programadas usan `@Scheduled` con consultas `FOR UPDATE SKIP LOCKED`, de modo que nunca procesen dos veces la misma solicitud. El vencimiento también se valida al leer, así que un retraso de la tarea no permite responder una solicitud vencida.

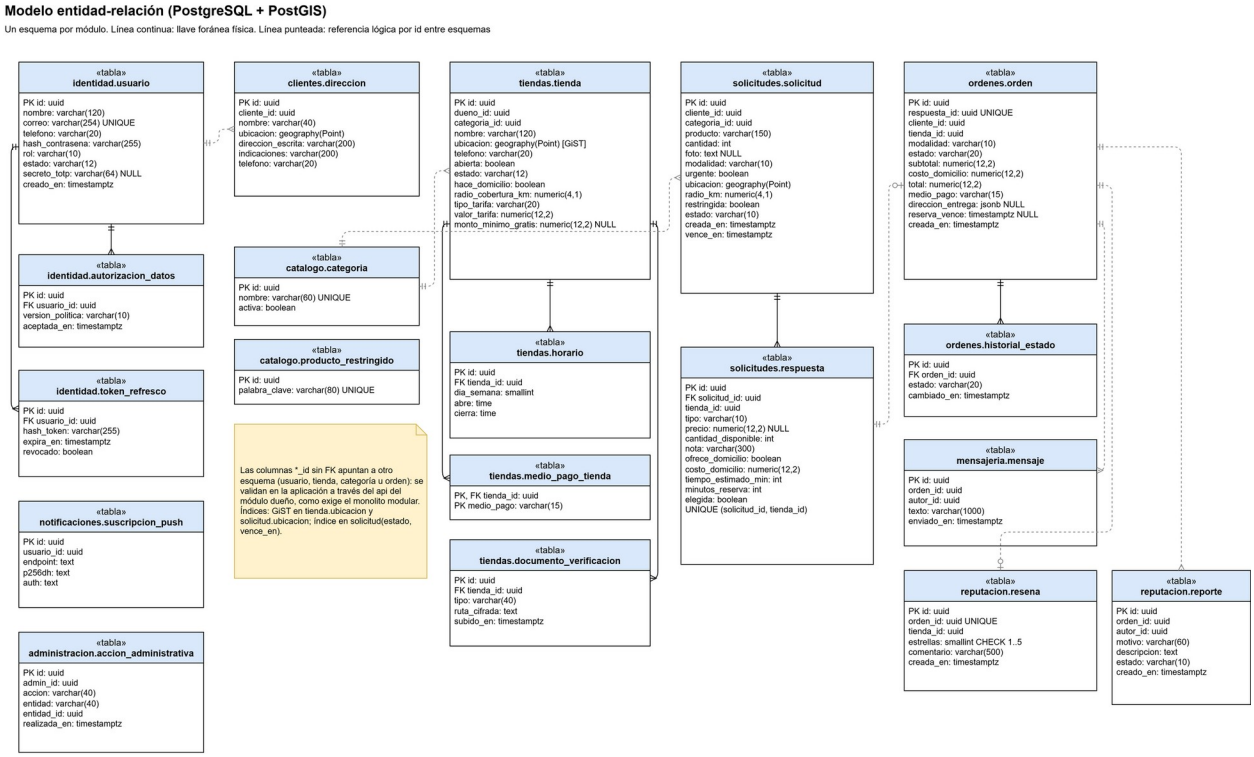
Vista de Datos

Hay una sola base PostgreSQL con un esquema por módulo (`identidad`, `tiendas`, `solicitudes`, `ordenes` y los demás). Entre esquemas no hay claves foráneas: un módulo guarda el identificador de un objeto de otro módulo y lo consulta por su `api`. Así, si un módulo se extrae en el futuro, sus datos se mueven sin romper restricciones.

- La ubicación de tiendas y solicitudes se guarda como `geography(Point, 4326)` con índice GiST, y la búsqueda de tiendas elegibles combina categoría, estado abierto, horario y radio en una sola consulta con `ST_DWithin`.
- La búsqueda de productos usa índices de trigramas (`pg_trgm`) y `unaccent`, que toleran errores de escritura y tildes.
- Los datos personales se minimizan: la tienda ve una distancia redondeada antes de ser elegida, nunca la ubicación exacta del cliente, y la dirección desaparece de su panel tras la entrega.
- Los documentos de verificación de tiendas se cifran con AES-GCM y las fotos se guardan sin metadatos EXIF, que pueden contener la ubicación GPS.

Figura 5

Modelo Entidad-Relación del Sistema



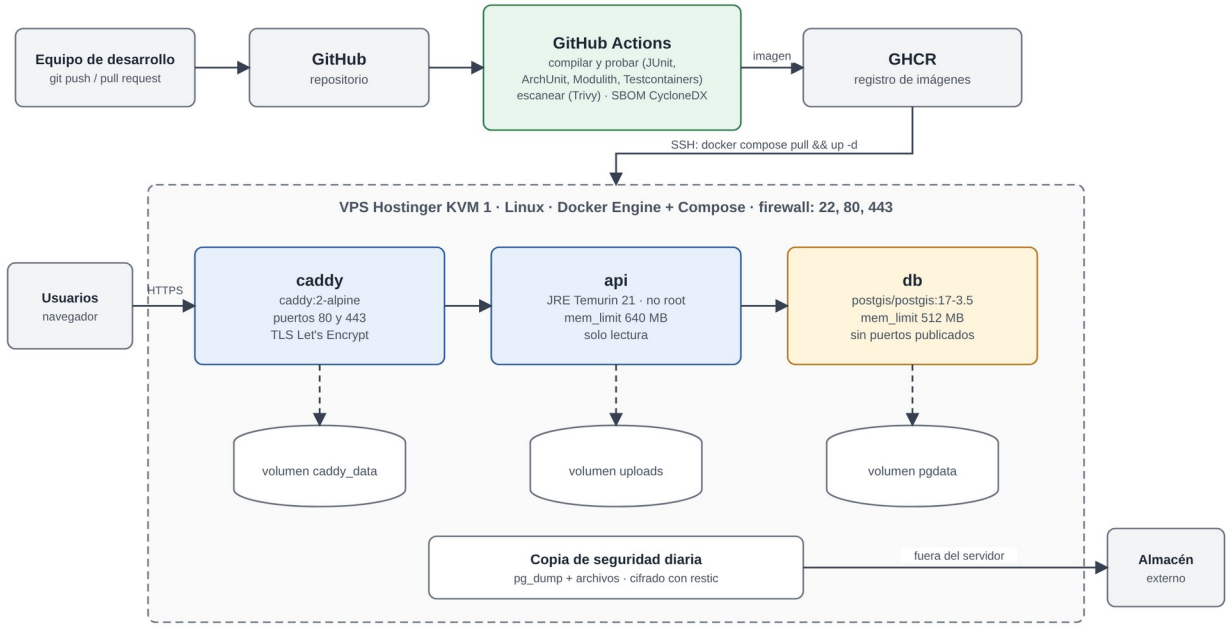
Nota. Exportado del diagrama 09-entidad-relacion.drawio del proyecto, donde puede ampliarse. Elaboración propia del equipo Byte Force.

Vista Física y Despliegue

El sistema corre en un VPS Linux con Docker Compose y tres contenedores: Caddy, la API y PostgreSQL con PostGIS (Figura 6). La imagen de la API se construye en GitHub Actions, nunca en el servidor, para no gastar su CPU ni su memoria.

Figura 6

Despliegue en Producción y Canal de Integración Continua



Nota. Basado en el diagrama 08-despliegue.drawio del proyecto. Las versiones de las imágenes se fijan a una versión exacta en el repositorio. Elaboración propia.

Tabla 13

Presupuesto de Memoria del Servidor

Proceso	RAM aproximada	Cómo se controla
Sistema operativo y Docker	250 MB	Linux de servidor mínimo, sin entorno gráfico
API Spring Boot	450 a 600 MB	Heap fijo de 256 a 384 MB; límite del contenedor de 640 MB
PostgreSQL + PostGIS	300 a 400 MB	`shared_buffers` de 256 MB, 30 conexiones como máximo; límite de 512 MB
Caddy	30 a 50 MB	—
Margen y caché del sistema	600 MB o más	Necesario para que PostgreSQL lea rápido del disco

Nota. Valores orientativos para 2 GB, que se miden durante las pruebas de rendimiento. Con 4 GB el margen es mayor.

El canal de integración y despliegue continuo tiene cuatro pasos: compilar y ejecutar las pruebas (unitarias, ArchUnit, Modulith e integración con Testcontainers); escanear dependencias e imagen con Trivy y generar el SBOM con CycloneDX; publicar la imagen en GitHub Container Registry; y desplegar por SSH con `docker compose pull && docker compose up -d`. En operación se hace un `pg_dump` diario más la carpeta de archivos, cifrado con restic y enviado fuera del servidor, con una prueba de restauración al mes; el firewall solo abre los puertos 22, 80 y 443 y SSH solo acepta llaves.

Diseño de la API REST

La API sigue el estilo REST definido por Fielding (2000): recursos identificados por URL, métodos HTTP con su semántica y comunicación sin estado. Las convenciones son:

- Versión en la URL (`/api/v1/...`) y JSON como formato.
- Documentación OpenAPI generada con springdoc, que el frontend usa como contrato.
- Errores en formato Problem Details (Nottingham et al., 2023), genéricos y sin trazas ni versiones internas.
- Paginación por cursor en historiales y validación con Bean Validation en todos los DTO de entrada.

Tabla 14

Recursos Principales de la API

Método y ruta	Rol	Propósito
`POST /api/v1/auth/registro`, `POST /api/v1/auth/sesion`	Público	Registro con autorización de datos e inicio de sesión
`POST /api/v1/solicitudes`	Cliente	Publicar lo que busca
`GET /api/v1/solicitudes/{id}/respuestas`	Cliente	Ver las respuestas, ordenables por precio o cercanía
`POST /api/v1/solicitudes/{id}/respuestas`	Tienda	Responder Sí, Parecido o No
`POST /api/v1/solicitudes/{id}/eleccion`	Cliente	Elegir una respuesta y crear la orden
`PATCH /api/v1/ordenes/{id}/estado`	Tienda, cliente	Avanzar o cancelar la orden según las transiciones válidas
`GET /api/v1/eventos`	Autenticado	Flujo SSE de avisos en tiempo real
`PATCH /api/v1/tiendas/{id}/apertura`	Tienda	Interruptor abierto o cerrado
`POST /api/v1/admin/tiendas/{id}/aprobacion`	Administrador	Aprobar o rechazar una tienda

Nota. Lista ilustrativa; el contrato definitivo es el documento OpenAPI publicado por la API en cada versión.

Patrones de Diseño y Principios SOLID

Cada patrón se usa donde resuelve un problema concreto del negocio, no para cumplir una lista (Gamma et al., 1994). Dos ajustes respecto a la propuesta inicial del equipo: en lugar de DAO se usa Repository, porque en la arquitectura hexagonal el dominio define la interfaz y el adaptador de Spring Data cumple el papel del DAO; y lo que se conocía como «Draper» es el patrón Decorator, mientras que el «Notificador» es Observer implementado con eventos de dominio.

Tabla 15

Patrones de Diseño Aplicados

Patrón	Dónde se usa	Problema que resuelve
Repository	`SolicitudRepository`, `OrdenRepository`	El dominio guarda y busca sin saber que existe JPA o SQL.
Puertos y adaptadores	Archivos, correo, push, reloj, pagos y geobúsqueda	Cambiar disco por S3 o agregar Wompi sin tocar los casos de uso.
Builder	Creación de Solicitud, Respuesta y Orden	Muchos campos opcionales con validación de invariantes al final.
Decorator	Caché sobre categorías y perfil de tienda; métricas y auditoría de casos de uso	Agregar comportamiento sin modificar la clase original.
Observer (eventos de dominio)	RespuestaRecibida avisa al cliente; OrdenResuelta habilita la reseña	Los módulos reaccionan sin conocerse entre sí.
Strategy	Tarifa de domicilio, canal de notificación, orden de respuestas	Variantes del negocio que crecen sin `if` encadenados.
State	Estados de Solicitud y Orden	Impedir transiciones inválidas.
Factory Method	Respuesta según el tipo Sí, Parecido o No	Cada tipo exige campos distintos.
Facade	Paquete `api` de cada módulo	Un punto de entrada estable para los demás módulos.
Specification	Criterios de tienda elegible	Reglas combinables que se prueban por separado y se traducen a una consulta.
Outbox transaccional	Registro de eventos de Spring Modulith	Un evento no se pierde si la aplicación se cae.

Los principios SOLID se reflejan así: **responsabilidad única**, con un caso de uso por acción y un controlador por recurso; **abierto y cerrado**, porque una nueva tarifa o canal es una nueva estrategia sin editar las existentes; **sustitución de Liskov**, porque cualquier implementación de `AlmacenArchivos` es intercambiable; **segregación de interfaces**, con puertos pequeños como `BuscadorTiendasCercanas`; e **inversión de dependencias**, porque los casos de uso dependen de interfaces del dominio y Spring inyecta los adaptadores.

Seguridad

El riesgo principal de esta aplicación no es un ataque sofisticado sino que una tienda vea datos de un cliente que no la eligió o que alguien abuse de las solicitudes. Por eso la autorización por objeto y los límites de uso van primero. La lista de verificación de referencia es OWASP

ASVS 5.0 nivel 2 (OWASP Foundation, 2025), complementada con el OWASP API Security Top 10 (OWASP Foundation, 2023) y las guías NIST SP 800.

Tabla 16

Controles de Seguridad

Control	Norma	Implementación
Autorización por objeto	OWASP API1	Cada caso de uso verifica que el recurso pertenece al usuario; la tienda solo ve datos del cliente si fue elegida y la orden está activa.
Exposición de datos	OWASP API3, Ley 1581	DTO de salida por rol; nunca se serializan entidades.
Roles y mínimo privilegio	NIST SP 800-53 (AC-6)	CLIENTE, TIENDA y ADMIN con Spring Security y `@PreAuthorize`.
Contraseñas	NIST SP 800-63B-4	Frase de mínimo 15 caracteres, sin reglas de composición, comparada con listas filtradas; hash Argon2id.
Segundo factor	NIST SP 800-63B-4	TOTP obligatorio para el administrador.
Límites de uso	OWASP API4	Bucket4j: intentos de inicio de sesión, 10 solicitudes por hora por cliente, respuestas por tienda.
Validación de entrada	OWASP Top 10 A05	Bean Validation, consultas parametrizadas y tamaños máximos.
Archivos subidos	OWASP File Upload	Tipo real por contenido, tamaño máximo, nombre aleatorio, fuera de la carpeta pública y sin EXIF.
Transporte y cabeceras	NIST SP 800-52 rev. 2	TLS 1.2 y 1.3 con Caddy, HSTS, CSP estricta y CORS solo para el dominio propio.
Secretos	NIST SP 800-57	Variables de entorno fuera del repositorio; llaves JWT con identificador para rotarlas.
Auditoría	NIST SP 800-92	Logs JSON sin datos personales; registro de aprobaciones, suspensiones y accesos a datos de clientes.
Desarrollo seguro	NIST SP 800-218	Trivy, OWASP Dependency-Check, SpotBugs con FindSecBugs y SBOM en cada build.
Contenedores	NIST SP 800-190	Imagen mínima, usuario no root, sistema de archivos de solo lectura, base sin puerto expuesto.

Nota. Elaboración propia con base en las normas citadas.

Rendimiento y Optimización

La medida que más ahorra es no agregar procesos: no hay Redis, broker de mensajes ni monitoreo pesado en el servidor. Además:

- **JVM:** heap explícito, Serial GC con 1 vCPU y Class Data Sharing para arrancar más rápido.

- **Hilos virtuales** (`spring.threads.virtual.enabled=true`): cada conexión SSE abierta cuesta casi nada.
- **Conexiones:** HikariCP con 5 a 10 conexiones y `open-in-view` desactivado.
- **Consultas:** proyecciones a DTO en los listados y `JOIN FETCH` contra el problema N+1.
- **Caché en memoria** con Caffeine para categorías, productos restringidos y perfiles de tienda, con tamaño máximo.
- **Frontend:** división de código por pantalla, archivos comprimidos y cacheados por Caddy, y fotos reducidas en el navegador antes de subirlas.
- **Opcional:** imagen nativa con GraalVM Community si la memoria no alcanza, compilada en CI y no en el servidor.

Pila Tecnológica y Licencias

Toda la pila usa licencias que permiten el uso comercial sin pago (Apache 2.0, MIT, BSD, EPL o PostgreSQL License). Las licencias deben revisarse de nuevo al fijar versiones, porque varios proyectos las han cambiado en los últimos años.

Tabla 17

Componentes de la Pila Tecnológica

Componente	Elección	Licencia
JDK	Eclipse Temurin 21 LTS	GPLv2 con Classpath Exception
Framework	Spring Boot 4, Spring Security, Spring Data JPA, Spring Modulith	Apache 2.0
ORM	Hibernate ORM + Hibernate Spatial	Apache 2.0
Base de datos	PostgreSQL 17 + PostGIS 3.5	PostgreSQL License; GPLv2
Migraciones	Flyway Community	Apache 2.0
Mapeo, caché, límites	MapStruct, Caffeine, Bucket4j	Apache 2.0
Documentación de API	springdoc-openapi	Apache 2.0
Pruebas	JUnit 5, Mockito, ArchUnit, Testcontainers	EPL 2.0, MIT, Apache 2.0, MIT
Frontend	React, Vite, React Router, TanStack Query	MIT
Mapas	Leaflet o MapLibre GL JS con OpenStreetMap	BSD-2 o BSD-3; ODbL para los datos
Proxy y TLS	Caddy 2	Apache 2.0

Componente	Elección	Licencia
Contenedores y CI	Docker Engine y Compose; GitHub Actions	Apache 2.0; gratuito con cuota
Seguridad en CI	Trivy, OWASP Dependency-Check, SpotBugs + FindSecBugs	Apache 2.0, Apache 2.0, LGPL
Copias de seguridad	restic	BSD-2

Tabla 18

Componentes que se Evitan

Producto	Problema	Alternativa
Oracle JDK	Licencia con posibles costos en producción	Eclipse Temurin
Redis 7.4 y posteriores	Cambió a RSAL y SSPL y luego agregó AGPL	Caché en memoria; si hace falta, Valkey (BSD)
MinIO	AGPLv3	Disco local detrás del puerto `AlmacenArchivos`
Liquibase 5	Cambió a Functional Source License	Flyway Community
react-leaflet	Hippocratic License, no es código abierto estándar	Leaflet directo o MapLibre
Elasticsearch	Licencias Elastic, SSPL o AGPL	Búsqueda de texto de PostgreSQL
Google Maps API	Costo por uso al superar la cuota	OpenStreetMap y enlaces «cómo llegar»

Nota. Las condiciones de uso comercial exigen conservar los avisos de licencia (página de licencias de terceros generada en el build) y mostrar «© OpenStreetMap contributors» en el mapa.

Verificación de la Arquitectura

La arquitectura se verifica con pruebas automáticas que corren en cada pull request, de manera que una violación de las reglas impide integrar el cambio. La Tabla 19 relaciona cada tipo de prueba con lo que protege.

Tabla 19

Pruebas que Protegen la Arquitectura

Tipo de prueba	Herramienta	Qué verifica	Responsable
Fronteras entre módulos	Spring Modulith	Uso solo de la `api` de otros módulos; ausencia de ciclos	Daniel
Reglas de capas	ArchUnit	El dominio no depende de Spring, JPA ni infraestructura	Daniel, Deiner
Unitarias de dominio	JUnit 5, Mockito	Reglas, máquinas de estado, especificaciones	Daniel, Deiner
Integración	Testcontainers con PostGIS	Consultas por radio, migraciones Flyway, eventos persistentes	Deiner
Aceptación de extremo a extremo	Playwright y escenarios Gherkin	Flujo publicar, responder, elegir y entregar	Yanimar
Rendimiento	k6	Tiempos de respuesta y consumo de memoria con carga de demostración	Francisco
Seguridad	OWASP ZAP, lista ASVS	Autorización por objeto, cabeceras, límites de uso	Francisco

Nota. Responsables según la asignación de roles del plan de proyecto.

Responsabilidades del Equipo sobre la Arquitectura

Tabla 20

Responsabilidades por Integrante

Integrante	Rol	Responsabilidad sobre la arquitectura
Daniel Montenegro Almendra	Arquitecto de software y Developer	Mantiene este documento y el registro de decisiones; crea la estructura base, el pipeline y los contenedores; revisa todo cambio que afecte las fronteras entre módulos.
Brayan Grajales Escarraga	Scrum Master y líder	Incluye en cada sprint las tareas técnicas (pruebas de concepto de PostGIS y SSE, deuda técnica) y vela por que la definición de terminado exija las pruebas de arquitectura en verde.
Wilson Javier Galindez Certuche	Product Owner y documentador	Mantiene la trazabilidad entre requisitos y módulos y actualiza los diagramas cuando cambia una decisión.
Monica Anyi Cajas Mellizo	UX y frontend	Construye la PWA, el service worker y el consumo del contrato OpenAPI; cuida la accesibilidad WCAG 2.1 AA y el peso de la aplicación.
Yanimar Burbano Piamba	QA funcional	Verifica los flujos y las transiciones de estado con pruebas de aceptación de extremo a extremo.
Francisco José Castro Muñoz	QA no funcional	Mide el rendimiento y la memoria frente al presupuesto y ejecuta las pruebas de seguridad.
Deiner Stiven Ramírez Vente	Developer backend	Desarrolla con Daniel los módulos backend, la persistencia PostGIS, las migraciones y las notificaciones.

Riesgos Arquitectónicos y Evolución

Tabla 21

Riesgos Arquitectónicos

ID	Riesgo	Impacto	Mitigación
RA-01	La memoria del servidor no alcanza con carga real.	Alto	Presupuesto de memoria medido con k6; imagen nativa como plan B.
RA-02	Curva de aprendizaje de PostGIS, SSE y push web.	Medio	Prueba de concepto en el Sprint 0 y ejemplos de referencia.
RA-03	Las fronteras entre módulos se erosionan por la prisa.	Medio	Pruebas de Modulith y ArchUnit que bloquean la integración.
RA-04	Un solo servidor es un punto único de falla.	Medio	Copias diarias cifradas fuera del servidor y restauración probada.
RA-05	Push web limitado en iPhone.	Bajo	SSE con la app abierta e instrucciones para instalar la PWA.
RA-06	Una librería cambia su licencia.	Bajo	Versiones fijas, SBOM y revisión de licencias al actualizar.

La arquitectura está pensada para crecer por etapas sin reescribirse: primero, más recursos para el mismo servidor; después, réplicas de lectura de PostgreSQL y almacenamiento de archivos compatible con S3 detrás del puerto existente; luego, la implementación del puerto `PasarelaPago` con Wompi o PayU y el plan de pago para tiendas; y, solo si un módulo como `notificaciones` lo exige por carga, su extracción como servicio con un broker de mensajes. Cada paso cambia adaptadores, no el dominio.

Conclusiones

El monolito modular con arquitectura hexagonal por módulo es la opción que mejor equilibra las restricciones del proyecto: cabe en un servidor de bajo costo, el equipo puede construirlo en cuatro sprints con las tecnologías que domina y conserva fronteras de negocio verificadas que permiten crecer. Los microservicios, Kubernetes o un broker de mensajes habrían multiplicado el costo y la complejidad sin resolver un problema que el MVP tenga.

Cada tecnología se eligió frente a alternativas concretas: Java y Spring Boot por la experiencia del equipo y su ecosistema integrado; PostgreSQL con PostGIS por las búsquedas

por radio con distancias reales; la PWA en React por tener un solo código sin costos de tiendas; SSE y push web por ser la forma más simple de avisar en tiempo real; y Docker Compose con Caddy por su operación sencilla y su TLS automático. Toda la pila es gratuita para uso comercial.

La arquitectura no queda solo en el papel: las reglas de los módulos y de las capas se comprueban con pruebas automáticas en cada cambio, y el presupuesto de memoria y los controles de seguridad se validan con pruebas de rendimiento y de seguridad antes de la entrega del 26 de noviembre de 2026.

Referencias

Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4.ª ed.). Addison-Wesley.

Cockburn, A. (2005). *Hexagonal architecture*. https://alistair.cockburn.us/hexagonal-architecture/

Congreso de la República de Colombia. (2012, 17 de octubre). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial No. 48.587.

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

Fowler, M. (2015, 3 de junio). *MonolithFirst*. https://martinfowler.com/bliki/MonolithFirst.html

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

International Organization for Standardization & International Electrotechnical Commission. (2023). *Systems and software engineering: Systems and software Quality Requirements and Evaluation (SQuaRE): Product quality model* (ISO/IEC 25010:2023).

International Organization for Standardization, International Electrotechnical Commission, & Institute of Electrical and Electronics Engineers. (2022). *Software, systems and enterprise: Architecture description* (ISO/IEC/IEEE 42010:2022).

Kruchten, P. B. (1995). The 4+1 view model of architecture. *IEEE Software, 12*(6), 42–50. https://doi.org/10.1109/52.469759

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2.ª ed.). O'Reilly Media.

Nottingham, M., Wilde, E., & Dalal, S. (2023). *Problem details for HTTP APIs* (RFC 9457). Internet Engineering Task Force. https://doi.org/10.17487/RFC9457

OWASP Foundation. (2023). *OWASP API Security Top 10*. https://owasp.org/API-Security/

OWASP Foundation. (2025). *Application Security Verification Standard 5.0*. https://owasp.org/www-project-application-security-verification-standard/

Souppaya, M., Morello, J., & Scarfone, K. (2017). *Application container security guide* (NIST Special Publication 800-190). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-190

Souppaya, M., Scarfone, K., & Dodson, D. (2022). *Secure software development framework (SSDF) version 1.1* (NIST Special Publication 800-218). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-218

Temoshok, D., Choong, Y.-Y., Galluzzo, R., LaSalle, M., Regenscheid, A., Proud-Madruga, D., Gupta, S., & Lefkovitz, N. (2025). *Digital identity guidelines: Authentication and authenticator management* (NIST Special Publication 800-63B-4). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-63B-4

Apéndice

Diagramas UML del Proyecto

Los diagramas fuente están en la carpeta `docs/diagramas` del repositorio, en formato draw.io, y pueden abrirse y exportarse con la aplicación de escritorio draw.io.

Tabla 22

Diagramas del Proyecto y Vista que Documentan

Archivo	Diagrama	Vista
`00-contexto.drawio`	Contexto del sistema	Escenarios
`01-casos-de-uso.drawio`	Casos de uso por actor	Escenarios
`02-clases-dominio.drawio`	Clases del dominio	Lógica
`03-secuencia-publicar-responder.drawio`	Secuencia: publicar y responder una solicitud	Procesos
`04-secuencia-elegir-orden.drawio`	Secuencia: elegir una respuesta y gestionar la orden	Procesos
`05-estados.drawio`	Estados de la solicitud y de la orden	Procesos
`06-actividades-flujo-principal.drawio`	Actividades del flujo principal	Procesos
`07-componentes.drawio`	Componentes: módulos del monolito	Lógica y desarrollo
`08-despliegue.drawio`	Despliegue en producción	Física
`09-entidad-relacion.drawio`	Modelo entidad-relación	Datos